Updated: Canvas Backup Tool

General description

When Canvas users download a full course export using the IMS Common Cartridge (.imscc) format, Canvas packages the entire course into a single compressed file. It is meant to transfer courses between institutions, not for user-friendly back-up systems. Even for users who understand how to extract the package, the content is not in a user-friendly or human navigable structure.

The Canvas Backup Tool will allow users, primarily instructors and instructional designers, to extract their downloaded IMS Common Cartridge file (.imscc) of their Canvas course into logically organized and named folders based on their course's module structure, so they can easily find and retrieve content outside of the Canvas environment.

The Canvas Backup Tool will be a desktop application (using Tkinter). Users will:

- Download their course IMS Common Cartridge file
- Open the Canvas Backup Tool application to find and select their .imscc file.
- Wait for the program to unzip the .imscc file, parse the xml, create a folder structure in the same folder as their .imscc file, and copies files into the logical folder structure that emulates the Canvas module setup as mapped in the xml.
- **Revisions**: Temp folder is deleted and a summary log is generated in the course backup folder

This program could be run as a script, if the user is comfortable running a command line interface (CLI) and adding the file path for the .imscc. The project should include external modules, such as:

- zipfile to handle .imscc extraction
- xml.etree.ElementTree for XML manifest parsing
- pathlib for local folder creation and file operations
- shutil for moving/copying files
- tkinter for GUI

AI Disclosure: I used the suggested prompt from the lecture notes, “Using python how would I perform X?” and got suggestions for Python modules, like the xml.etree.ElementTree and shutil.

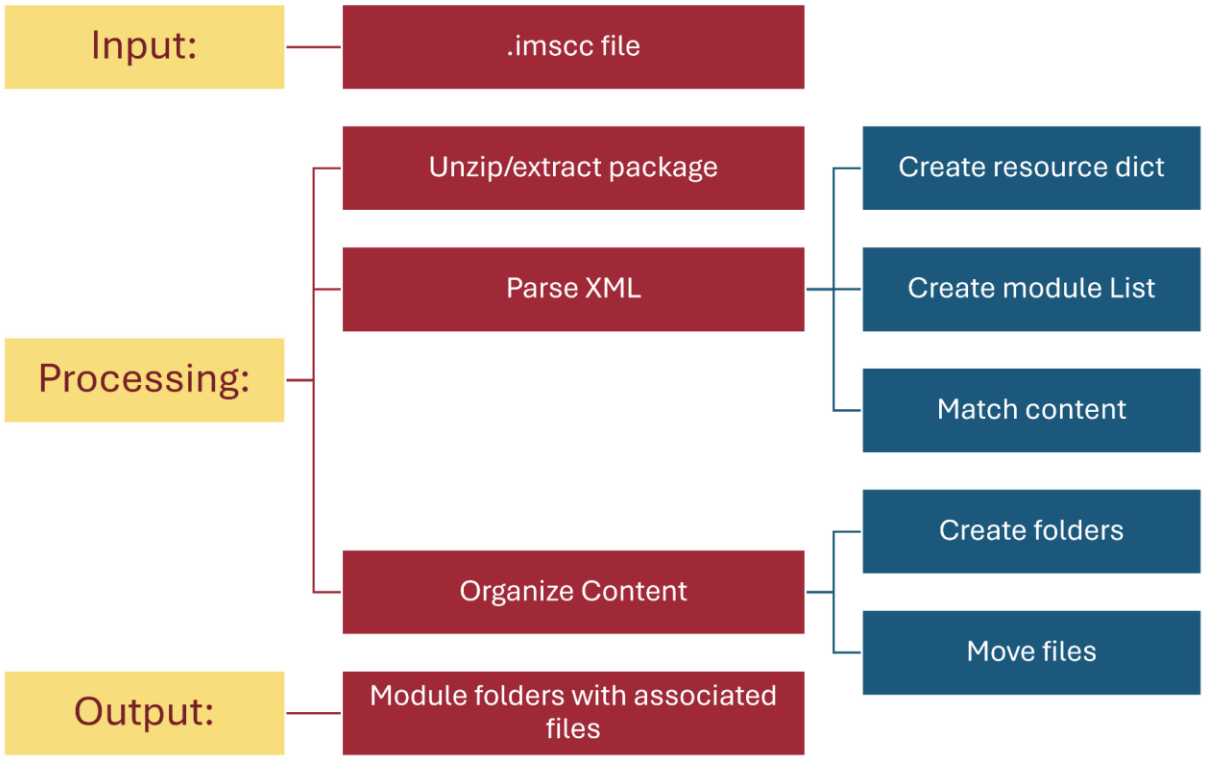
Task vignettes / User activity flow

An instructor wants to backup their course in case Canvas is hacked, so they can access their assignments and exams to keep class running. They download their course export (.imscc) from

Canvas, launch the Canvas backup tool, select their file, then their content is moved to module folders in the same folder as their .imscc file.

1. The instructor visits their Canvas course, goes to course settings, requests an export of their course, and downloads the .imscc file.
2. The instructor launches the desktop application (tkinter GUI) and browses for their file.
    a. Alternatively, they can launch CLI, run the python script, then type in the file path to their .imscc (pathlib)
3. The program/script unzips the package (zipfile), parses the .xml (xml.etree.ElementTree or Beautifulsoup) for organization and resources, and creates a resource dict and module list that is matched by the xml identifiers.
4. Module folders are created (pathlib) and the identified module content is copied into module folders (shutil).
5. The instructor can now browse through their content and easily find their backup files (uploaded course files and HTML pages of content, assignments, exams, etc.)

Technical flow



Revisions: Temp folder is deleted and a summary log is generated in the course backup folder

Self Assessment

After working through the spec, what was the biggest or most unexpected change you had to make from your sketch?

My original idea was too complicated, so I pulled it back to extraction and parsing. I originally wanted to convert, merge files, etc.

Revision: I am updating my already revised project specs, but this was a big change from the original!

How confident do you feel that you can implement the spec as it's written right now?

I used the suggested Copilot prompt in our lecture notes and feel like I can pull off extracting the .imscc package, the file operations, but I need to look back at parsing the manifest xm, and spend time looking through the xml myself to understand the identifiers so I can build an index using dictionaries and test it. Copilot suggested using lists to build the module map, and it looks like they do map to resource identifiers under <organizations> when I look at the XML, so that seems doable.

I think I can technically pull it off because I've done related class exercises, but in reality, I'll have to reference lecture materials a lot, debug, and maybe ask Copilot for feedback on my code. I think that python modules will do heavy lifting for the beginning but I am worried about mapping everything correctly, especially if I'm looking at example courses that are "perfect" for mapping. I probably need to look at a variety of course export packages (.imscc) to test this against.

What is the biggest potential problem that you NEED to solve (or you'll fail)?
Accurately parsing the Canvas XML manifest to resolve identifiers without losing items.

Revision: Missing quizzes and discussions, but have added a summary log to help user ID missing items (and what was successfully found and copied over).
Nice to have: Figure out how to copy over quizzes

What parts are you least familiar with and might need my help?
There may be edge cases in the imsmanifest.xml where module items reference something not identified in the package.